Mini-Project 4: Moving Beyond Linearity

EN.553.434/634: Elements of Statistical Learning

Jonathan Ma

1. Polynomial regression: a nonlinear simulation

Generate one training set D with $n = 80$ and one independent test set Te with $n_{\text{Te}} = 5,000$ from

$$X \sim \text{Unif}(0, 1), \qquad Y = \sin(2\pi X) + \varepsilon, \qquad \varepsilon \sim N(0, 0.5^2),$$

where X and ε are independent. Use a fixed random seed. Under squared-error loss, the true regression function is

$$f^*(x) = E[Y \mid X = x] = \sin(2\pi x).$$

Plot the training observations together with $f^*(x)$ on $[0, 1]$. Then fit least-squares polynomial regressions of degrees $d = 1, 3, 5, 10$, using the training data only. Display the four fitted curves and the true regression function (use separate panels if necessary).

Recall that the degree-d polynomial model class is

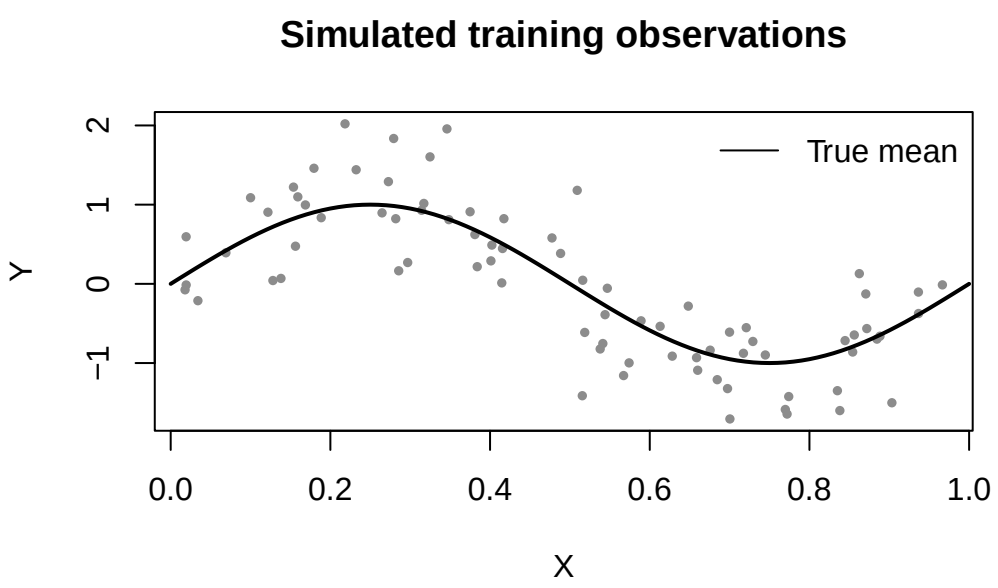
$$\mathcal{F}_d = \left\{\beta_0 + \beta_1 x + \cdots + \beta_d x^d\right\}.$$

Even the best predictor $f^*_{\mathcal{F}_d}$ in this class does not exactly equal the sine function for any finite d; there is no need to compute $f^*_{\mathcal{F}_d}$ in this project.

```
set.seed(434634)
library(ISLR2)
library(splines)
library(glmnet)
library(mgcv)
data("Auto")
mse <- function(y, pred) mean((y - pred)^2)
poly_features <- function(x, d) {
  z <- outer(x, seq_len(d), `^`)
  colnames(z) <- paste0("x", seq_len(d))
  z
}
# Fixed folds for fair comparisons, constructed only from training observations.
make_folds <- function(n, k = 5) sample(rep(seq_len(k), length.out = n))
```

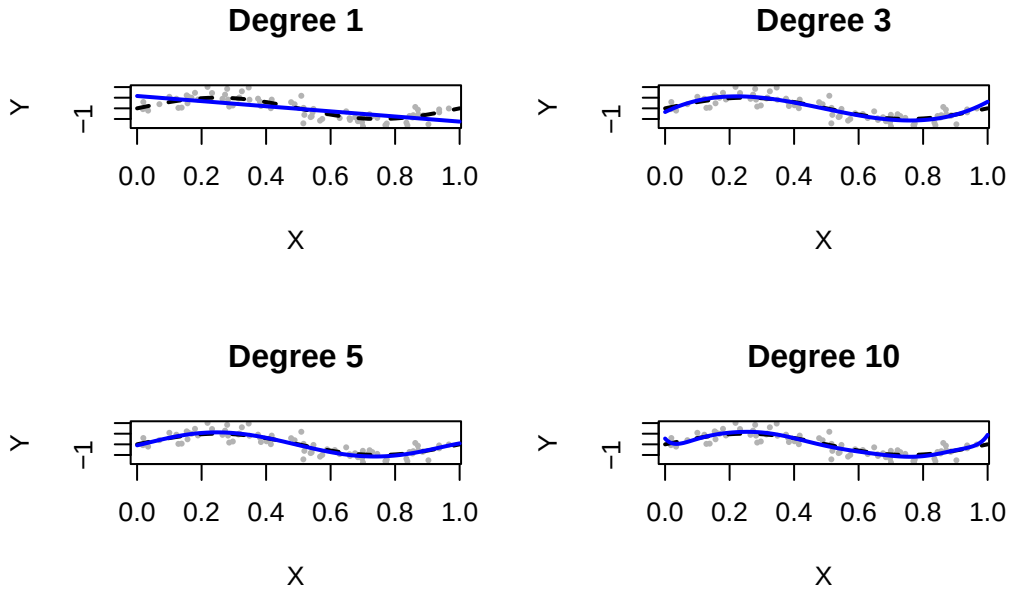
```r
cv_lm <- function(data, degrees, formula_fn, folds) {
  vapply(degrees, function(d) {
    mean(vapply(seq_len(5), function(j) {
      tr <- data[folds != j, , drop = FALSE]
      va <- data[folds == j, , drop = FALSE]
      fit <- lm(formula_fn(d), data = tr)
      mse(va$y, predict(fit, newdata = va))
    }, numeric(1)))
  }, numeric(1))
}
n <- 80; n_test <- 5000; sigma <- 0.5
x_train <- runif(n); y_train <- sin(2*pi*x_train) + rnorm(n, sd=sigma)
x_test <- runif(n_test); y_test <- sin(2*pi*x_test) + rnorm(n_test, sd=sigma)
f_true <- function(x) sin(2*pi*x)
sim_train <- data.frame(x=x_train, y=y_train)
sim_test <- data.frame(x=x_test, y=y_test)
sim_grid <- data.frame(x=seq(0, 1, length.out=400))
sim_folds <- make_folds(n)
```

```r
plot(y ~ x, sim_train, pch=16, cex=.65, col="gray55",
     main="Simulated training observations", xlab="X", ylab="Y")
curve(f_true(x), from=0, to=1, add=TRUE, col="black", lwd=2)
legend("topright", "True mean", col="black", lty=1, bty="n")
```




```r
poly_degrees <- c(1,3,5,10)
poly_fits <- lapply(poly_degrees, function(d) lm(y ~ poly(x,d,raw=TRUE), data=sim_train))
#| fig-width: 9
#| fig-height: 7
par(mfrow=c(2,2))
for (j in seq_along(poly_degrees)) {
  plot(y ~ x, sim_train, pch=16, cex=.5, col="gray70",
```

```
       main=paste("Degree", poly_degrees[j]), xlab="X", ylab="Y")
  lines(sim_grid$x, f_true(sim_grid$x), lwd=2, lty=2)
  lines(sim_grid$x, predict(poly_fits[[j]], sim_grid), col="blue", lwd=2)
}
```




```
par(mfrow=c(1,1))
```

Q1. Why is the degree-1 polynomial model inadequate for this regression function? Is polynomial regression still a linear model in the coefficients?

> A straight line cannot reproduce the oscillation and changing slope of $\sin(2\pi x)$. Polynomial regression is still linear in its coefficients β_j, despite being nonlinear in x.

Q2. Compare the fitted curves for $d = 1, 3, 5, 10$. What changes as the polynomial degree increases, especially near the ends of the interval?

> Increasing the polynomial degree expands the model space, allowing the fitted curve to approximate the sine function more closely. Higher degrees can also increase sensitivity to noise and produce unnecessary oscillation, particularly near the endpoints. These are risks of increased flexibility, not guaranteed features of every higher-degree fit.

2. Choosing polynomial degree

Using only the training data D, use 5-fold cross-validation to compare polynomial degrees

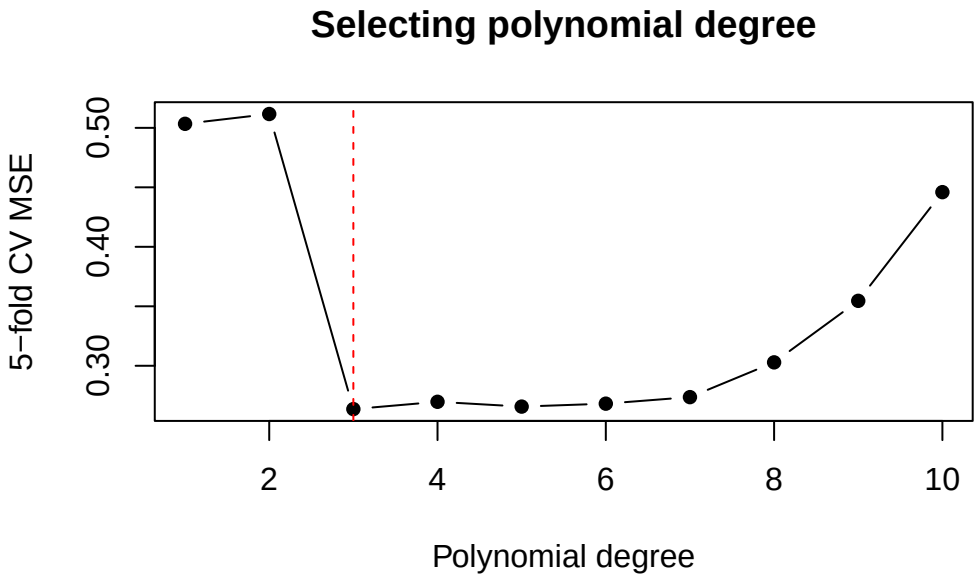
$$d = 1, 2, \ldots, 10.$$

Plot mean cross-validation squared error against d and select the degree with the smallest CV error. Fit the selected polynomial on all of D, then report its mean squared error on the independent test set Te:

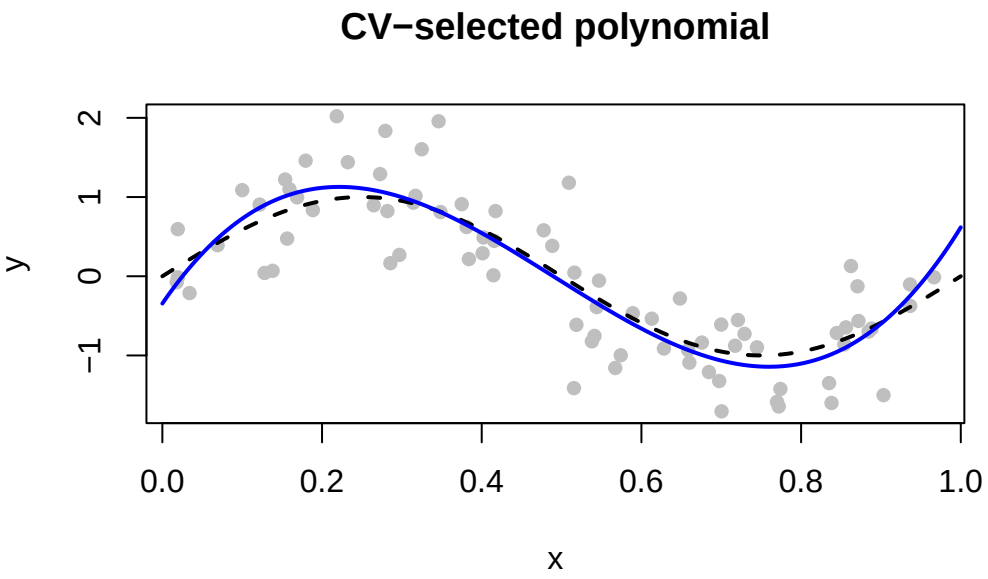
$$\widehat{R}_{\mathrm{Te}}(\hat{f}_D) = \frac{1}{n_{\mathrm{Te}}} \sum_{i \in \mathrm{Te}} \left(Y_i - \hat{f}_D(X_i)\right)^2 .$$

Keep the test set unchanged for all comparisons in Parts 1–3; do **not** use it to choose the polynomial degree or tuning parameters.

```
sim_cv <- cv_lm(sim_train, 1:10,
                function(d) y ~ poly(x,d,raw=TRUE), sim_folds)
sim_best_d <- which.min(sim_cv)
sim_best_fit <- lm(y ~ poly(x,sim_best_d,raw=TRUE), data=sim_train)
sim_best_test <- mse(sim_test$y, predict(sim_best_fit, sim_test))
plot(1:10, sim_cv, type="b", pch=16, xlab="Polynomial degree",
     ylab="5-fold CV MSE", main="Selecting polynomial degree")
abline(v=sim_best_d, lty=2, col="red")
```




```
plot(y ~ x, sim_train, pch=16, col="gray75", main="CV-selected polynomial")
lines(sim_grid$x, f_true(sim_grid$x), lty=2, lwd=2)
lines(sim_grid$x, predict(sim_best_fit, sim_grid), col="blue", lwd=2)
```




```
cat("Selected degree:",sim_best_d,"; test MSE:",round(sim_best_test,4),"\n")
```

```
Selected degree: 3 ; test MSE: 0.2659
```

Q3. Which degree does cross-validation select? How does the selected fit compare visually with the true regression function?

> Five-fold CV selects degree 3. Its fitted curve can be compared with the dashed true sine curve in the plot above; discrepancies reflect finite-sample estimation error and polynomial approximation error.

Q4. Why can increasing polynomial degree improve approximation to f^* but worsen prediction from a finite training sample?

> Higher-degree polynomial spaces approximate more functions and reduce approximation bias, but estimating more coefficients from only 80 observations increases variance and can overfit noise. Test error therefore need not decrease with degree.

3. Ridge and lasso with polynomial features

Continue with the same training and test sets. Use the degree-10 polynomial features

$$X,\ X^2,\ \ldots,\ X^{10}.$$

Fit (a) ordinary least squares, (b) ridge regression, and (c) lasso regression using these features. For ridge and lasso, use 5-fold cross-validation **on** D to choose λ. Display the CV squared-error curve against λ for each method; a logarithmic horizontal axis is recommended.

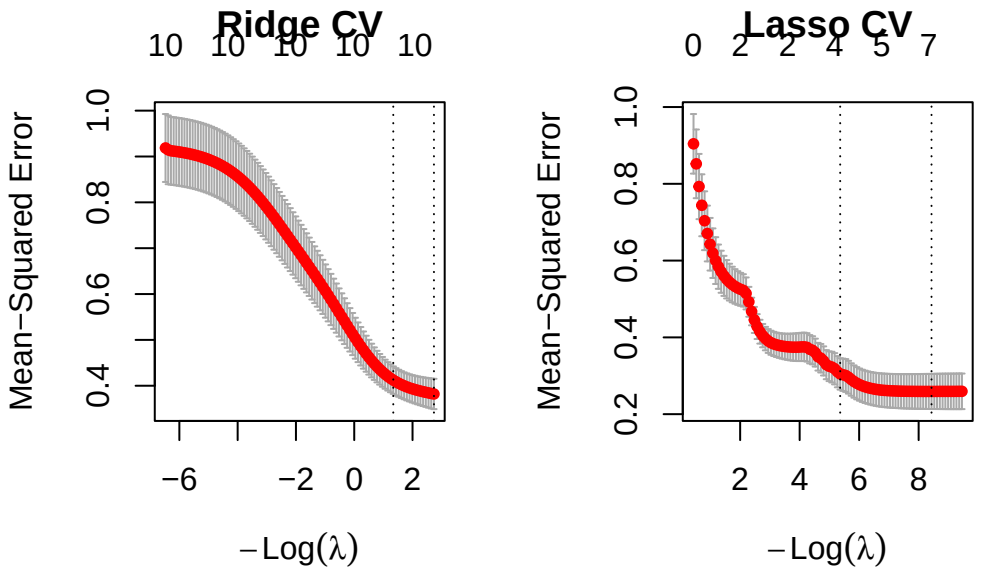
Let $z_j(x)$ denote the standardized version of x^j (using the appropriate training data), and write $f_\beta(x) = \beta_0 + \sum_{j=1}^{10} \beta_j z_j(x)$. Recall the penalized objectives (with the intercept unpenalized):

$$\text{Ridge:}\quad \frac{1}{n}\sum_{i=1}^{n}\big(Y_i - f_\beta(X_i)\big)^2 + \lambda \sum_{j=1}^{10} \beta_j^2,$$

$$\text{Lasso:} \quad \frac{1}{n}\sum_{i=1}^{n}(Y_i - f_\beta(X_i))^2 + \lambda\sum_{j=1}^{10}|\beta_j|,$$

Note: Standardize the polynomial features for penalized regression. Any standardization must be learned from the training portion of each CV fold and applied to its validation portion. Do not penalize the intercept. Different software packages may use different conventions for the numerical value of λ.

```
X_sim <- poly_features(x_train,10)
X_sim_test <- poly_features(x_test,10)
X_sim_grid <- poly_features(sim_grid$x,10)
# Use the same folds for ridge and lasso. glmnet standardizes within CV fits.
ridge_cv <- cv.glmnet(X_sim, y_train, alpha=0, nfolds=5,
                      foldid=sim_folds, standardize=TRUE)
lasso_cv <- cv.glmnet(X_sim, y_train, alpha=1, nfolds=5,
                      foldid=sim_folds, standardize=TRUE)
ols10 <- lm(y ~ poly(x,10,raw=TRUE), data=sim_train)
par(mfrow=c(1,2))
plot(ridge_cv, main="Ridge CV")
plot(lasso_cv, main="Lasso CV")
```




```
par(mfrow=c(1,1))
cat("Ridge lambda.min:",ridge_cv$lambda.min,"\n")
```

```
Ridge lambda.min: 0.0649103
```

```
cat("Lasso lambda.min:",lasso_cv$lambda.min,"\n")
```

```
Lasso lambda.min: 0.0002175535
```

Q5. Why is standardization particularly important when ridge or lasso is applied to $X, X^2, \ldots, X^{10}$?

The scales and variances of $X, X^2, \ldots, X^{10}$ differ substantially. Standardization makes the penalties comparable across coefficients and prevents feature scale, rather than

predictive value, from determining shrinkage.

Q6. Which λ values are selected for ridge and lasso? What do their CV curves indicate about very small and very large penalties?

> CV selects ridge penalty 0.06491 and lasso penalty 2.176×10^{-4} (using `glmnet` conventions). Very small penalties approach an unregularized, potentially high-variance fit; very large penalties overshrink coefficients and can underfit.

Compare the ridge and lasso coefficient estimates. Report which polynomial terms, if any, have coefficients exactly zero (or zero up to numerical tolerance). Then plot, on the same axes, f^* and the fitted curves from the degree-10 least-squares, ridge, and lasso models.

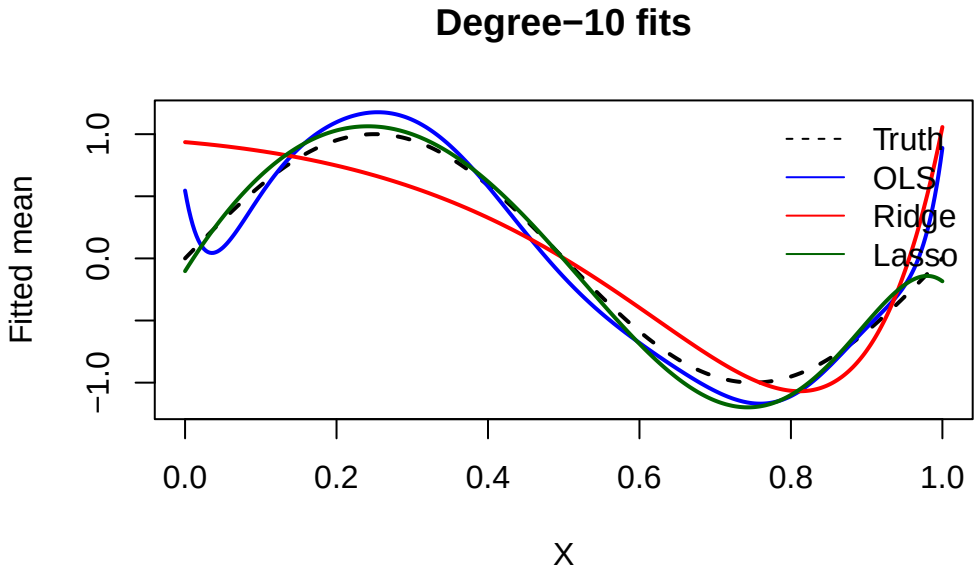
```
ridge_coef <- as.numeric(coef(ridge_cv, s="lambda.min"))[-1]
lasso_coef <- as.numeric(coef(lasso_cv, s="lambda.min"))[-1]
coef_table <- data.frame(term=paste0("x^",1:10), ridge=ridge_coef,
                         lasso=lasso_coef)
print(coef_table, row.names=FALSE)
```

```
 term      ridge         lasso
  x^1 -0.5291813   9.738251867
  x^2 -1.7284219 -20.427379429
  x^3 -1.5250301   0.000000000
  x^4 -0.9274930   0.069312496
  x^5 -0.2753436   2.248227624
  x^6  0.3041856  17.082355161
  x^7  0.7721559   0.000000000
  x^8  1.1254885  -0.003345879
  x^9  1.3739938  -0.464213779
 x^10  1.5312400  -8.324927944
```

```
lasso_zero <- coef_table$term[abs(lasso_coef) < 1e-8]
cat("Lasso zero terms:",if(length(lasso_zero)) paste(lasso_zero,collapse=", ") else "none","\n"
```

```
Lasso zero terms: x^3, x^7
```

```
plot(sim_grid$x, f_true(sim_grid$x), type="l", lty=2, lwd=2,
     ylim=range(c(f_true(sim_grid$x),predict(ols10,sim_grid),
                  predict(ridge_cv,newx=X_sim_grid,s="lambda.min"),
                  predict(lasso_cv,newx=X_sim_grid,s="lambda.min"))),
     xlab="X", ylab="Fitted mean", main="Degree-10 fits")
lines(sim_grid$x,predict(ols10,sim_grid),col="blue",lwd=2)
lines(sim_grid$x,as.numeric(predict(ridge_cv,newx=X_sim_grid,s="lambda.min")),col="red",lwd=2)
lines(sim_grid$x,as.numeric(predict(lasso_cv,newx=X_sim_grid,s="lambda.min")),col="darkgreen",l
legend("topright",c("Truth","OLS","Ridge","Lasso"),
       lty=c(2,1,1,1),col=c("black","blue","red","darkgreen"),bty="n")
```



Q7. How do ridge and lasso differ in their treatment of the polynomial coefficients? Which produces a sparse solution, if any?

> Ridge shrinks coefficients continuously toward zero, whereas lasso can set coefficients exactly to zero. At the selected penalty, lasso zeroes x^3, x^7; ridge typically retains all terms.

Evaluate the following four models on the *same* independent test set: (a) the CV-selected polynomial from Part 2, (b) degree-10 ordinary least squares, (c) CV-selected ridge, and (d) CV-selected lasso. Present their test MSEs in one table.

```
sim_results <- data.frame(
  Model = c("CV polynomial", "Degree-10 OLS", "Ridge", "Lasso"),
  Test_MSE = c(
    sim_best_test,
    mse(y_test, predict(ols10, sim_test)),
    mse(y_test, as.numeric(predict(ridge_cv, newx = X_sim_test, s = "lambda.min"))),
    mse(y_test, as.numeric(predict(lasso_cv, newx = X_sim_test, s = "lambda.min")))
  )
)

print(sim_results, row.names = FALSE)
```

```
         Model  Test_MSE
 CV polynomial 0.2658911
 Degree-10 OLS 0.2677898
         Ridge 0.3294336
         Lasso 0.2566126
```

```
sim_winner <- sim_results$Model[which.min(sim_results$Test_MSE)]
```

Q8. Which model has the smallest test MSE in your experiment? Did regularization help relative to unpenalized degree-10 regression?

The lowest observed test MSE belongs to **Lasso**. Relative to degree-10 OLS, ridge does not improve test MSE, and lasso improves it.

Q9. If lasso selects only a few polynomial terms, does this mean it has identified the true functional form of f^*? Explain.

No. The true function is sinusoidal, not a finite polynomial. Lasso selects a sparse approximation in a highly correlated polynomial basis; its selected terms are not evidence of the true generating functional form.

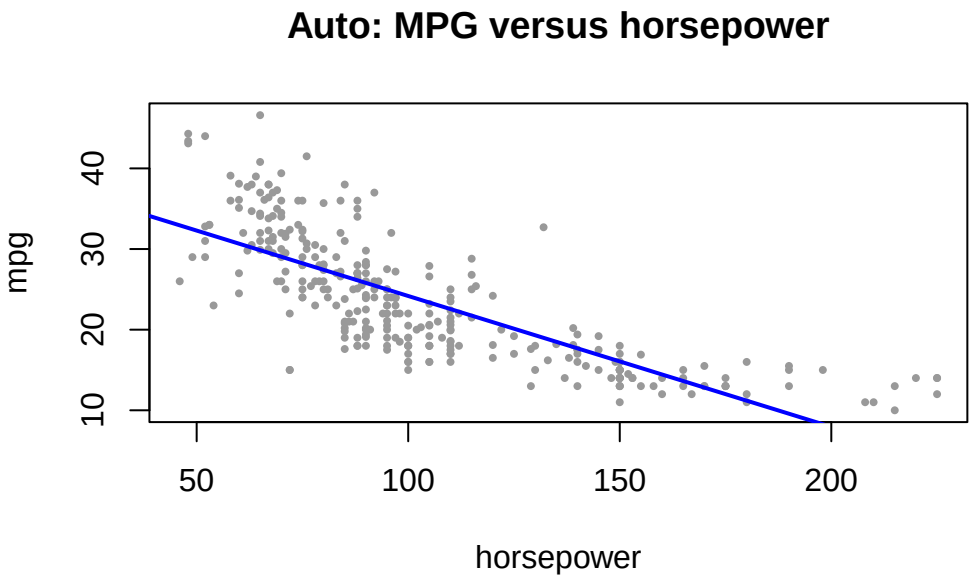
4. Auto data: linear and polynomial regression

Now use the `Auto` dataset from the `ISLR2` package (or its Python equivalent). The response is `mpg`; initially use `horsepower` as the only predictor.

Randomly reserve 20% of observations as a test set Te and use the remaining 80% for training D. Keep this split for the rest of the project. Perform all model selection using the training set only.

Plot `mpg` against `horsepower` and fit a simple linear regression using D.

```
set.seed(434635)
Auto <- na.omit(ISLR2::Auto)
train_idx <- sample(seq_len(nrow(Auto)), size=floor(.8*nrow(Auto)))
auto_train <- Auto[train_idx,,drop=FALSE]
auto_test <- Auto[-train_idx,,drop=FALSE]
# Keep original column names for later model formulas; y is CV response alias.
auto_train$y <- auto_train$mpg
auto_test$y <- auto_test$mpg
auto_folds <- make_folds(nrow(auto_train))
auto_linear <- lm(mpg ~ horsepower, data=auto_train)
plot(mpg ~ horsepower, data=auto_train, pch=16, cex=.55,
     col="gray60", main="Auto: MPG versus horsepower")
abline(auto_linear,col="blue",lwd=2)
```

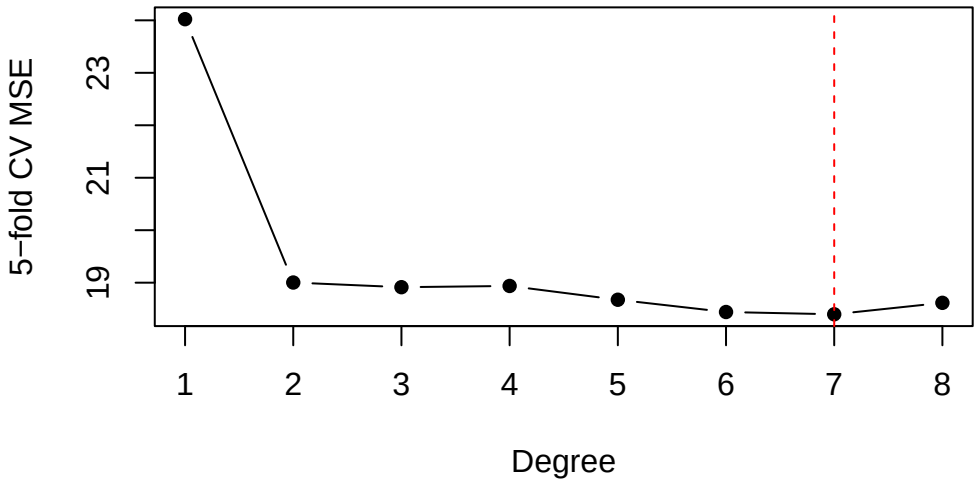


Q10. Does the scatterplot suggest that a straight-line relationship is adequate? Explain briefly.

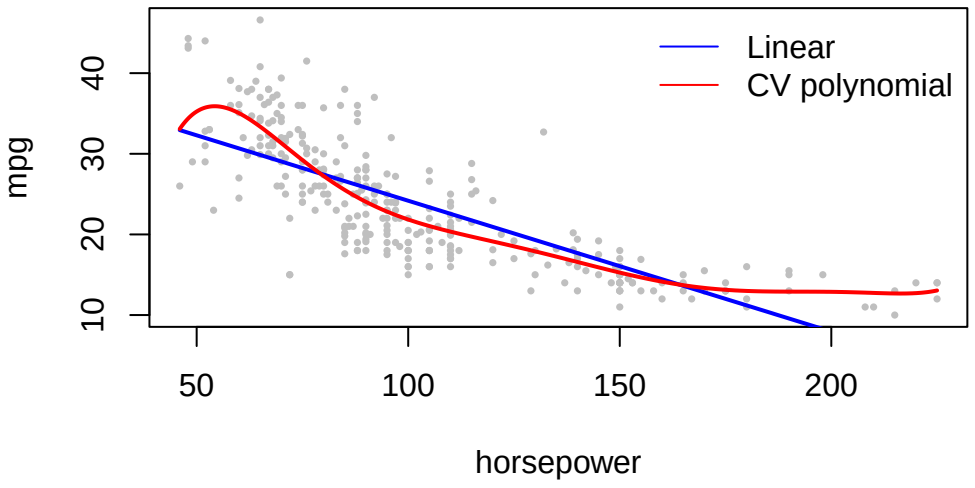
> The scatterplot suggests a nonlinear decreasing relationship: MPG falls rapidly at lower horsepower and tends to flatten at higher horsepower. A straight line cannot capture this changing slope.

Fit polynomial regressions of degrees $1, \ldots, 8$, choosing the degree by 5-fold CV on D. Plot the CV errors and display the selected fit together with the simple linear fit and the data.

```
auto_poly_cv <- cv_lm(auto_train,1:8,
  function(d) y ~ poly(horsepower,d,raw=TRUE),auto_folds)
auto_best_d <- which.min(auto_poly_cv)
auto_poly <- lm(mpg ~ poly(horsepower,auto_best_d,raw=TRUE),data=auto_train)
hp_grid <- data.frame(horsepower=seq(min(auto_train$horsepower),
                                     max(auto_train$horsepower),length.out=400))
plot(1:8,auto_poly_cv,type="b",pch=16,xlab="Degree",ylab="5-fold CV MSE")
abline(v=auto_best_d,col="red",lty=2)
```




```
plot(mpg ~ horsepower,auto_train,pch=16,cex=.5,col="gray75")
lines(hp_grid$horsepower,predict(auto_linear,hp_grid),col="blue",lwd=2)
lines(hp_grid$horsepower,predict(auto_poly,hp_grid),col="red",lwd=2)
legend("topright",c("Linear","CV polynomial"),col=c("blue","red"),lty=1,bty="n")
```



```r
cat("Auto selected polynomial degree:",auto_best_d,"\n")
```

```
Auto selected polynomial degree: 7
```

Q11. Which degree is selected? What relationship is the polynomial model capturing that the straight-line model may miss?

> CV selects degree 7. The polynomial allows curvature in the MPG–horsepower relationship, especially the steeper decline at low horsepower and flatter behavior at high horsepower.

5. Regression splines and natural splines

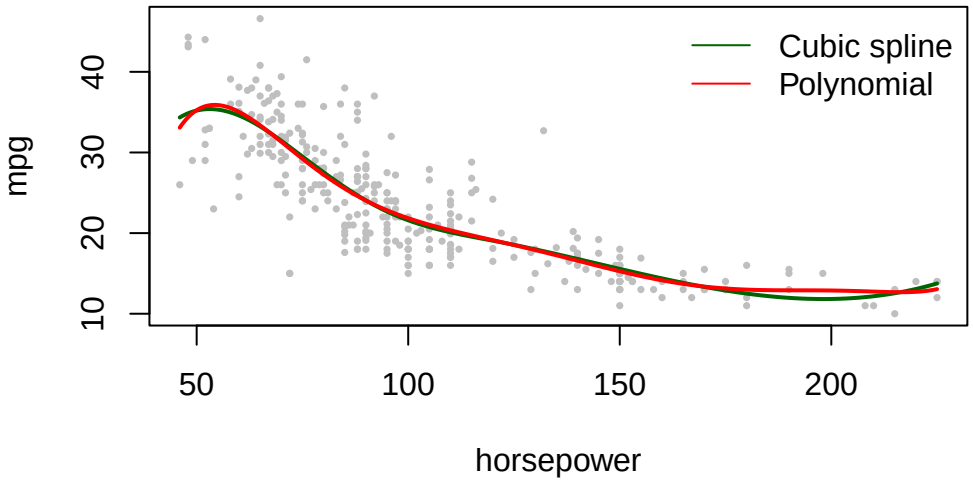
Continue predicting `mpg` from `horsepower` alone.

First, fit an ordinary cubic regression spline with interior knots at the 25th, 50th, and 75th percentiles of **training-set** `horsepower` (for example, using `bs()` in R). Plot the fitted spline along with the selected polynomial from Part 4.

```r
hp_knots <- as.numeric(quantile(auto_train$horsepower,c(.25,.5,.75)))
# Boundary knots fixed using the training data, and reused at prediction.
hp_bounds <- range(auto_train$horsepower)
bs_formula <- mpg ~ bs(horsepower,degree=3,knots=hp_knots,Boundary.knots=hp_bounds)
bs_fit <- lm(bs_formula,data=auto_train)
cat("Cubic spline basis functions:",ncol(model.matrix(bs_fit))-1,"\n")
```

```
Cubic spline basis functions: 6
```

```r
plot(mpg ~ horsepower,auto_train,pch=16,cex=.5,col="gray75")
lines(hp_grid$horsepower,predict(bs_fit,hp_grid),col="darkgreen",lwd=2)
lines(hp_grid$horsepower,predict(auto_poly,hp_grid),col="red",lwd=2)
legend("topright",c("Cubic spline","Polynomial"),
       col=c("darkgreen","red"),lty=1,bty="n")
```



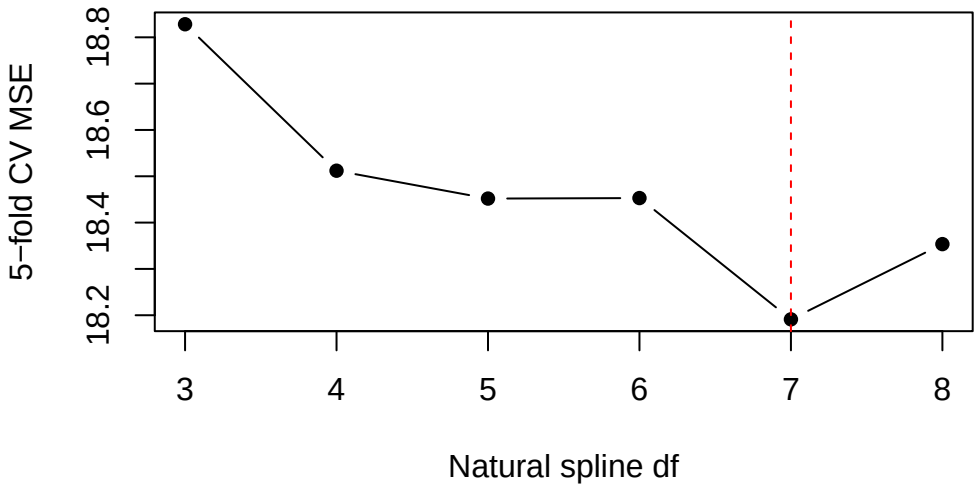
Q12. How many spline basis functions are used, excluding the overall regression intercept? How do the knots allow the fit to behave differently from a single global polynomial?

There are **6** cubic spline basis functions, excluding the intercept. The three interior knots allow different cubic pieces to adapt locally while maintaining continuity of the fitted curve and its first two derivatives.

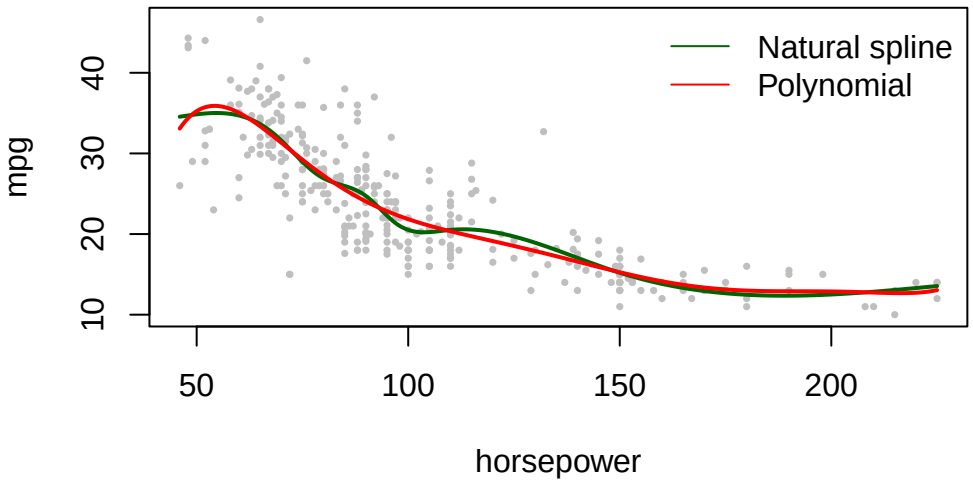
Next, fit natural cubic splines with degrees of freedom $3, 4, \ldots, 8$, selecting the degrees of freedom by 5-fold CV on D (for example, using `ns()` in R). Plot the CV errors and the selected natural spline together with the selected polynomial.

Note: When using basis functions in cross-validation, construct the basis using the training portion of the fold, and apply that *same fitted basis* to its validation portion. This also applies when predicting on the held-out test set.

```
# ns() is fitted separately on each CV training fold by lm(); predict()
# reuses the fold's knots and boundary knots for validation observations.
ns_cv <- cv_lm(auto_train,3:8,
  function(d) y ~ ns(horsepower,df=d),auto_folds)
ns_best_df <- (3:8)[which.min(ns_cv)]
ns_fit <- lm(mpg ~ ns(horsepower,df=ns_best_df),data=auto_train)
plot(3:8,ns_cv,type="b",pch=16,xlab="Natural spline df",ylab="5-fold CV MSE")
abline(v=ns_best_df,col="red",lty=2)
```




```
plot(mpg ~ horsepower,auto_train,pch=16,cex=.5,col="gray75")
lines(hp_grid$horsepower,predict(ns_fit,hp_grid),col="darkgreen",lwd=2)
lines(hp_grid$horsepower,predict(auto_poly,hp_grid),col="red",lwd=2)
legend("topright",c("Natural spline","Polynomial"),
       col=c("darkgreen","red"),lty=1,bty="n")
```




```
cat("Selected natural spline df:",ns_best_df,"\n")
```

```
Selected natural spline df: 7
```

Q13. What degrees of freedom does CV select for the natural spline? Compare its shape with the selected polynomial, particularly near the boundaries. What restrictions distinguish a natural cubic spline from an ordinary cubic spline?

> CV selects **7** basis degrees of freedom (excluding the overall intercept). Both the natural spline and the selected polynomial capture curvature in the MPG–horsepower relationship. The natural spline permits local changes in curvature while the polynomial uses a single global expression. Unlike an ordinary cubic spline, the natural spline is linear beyond its boundary knots, which can reduce unstable tail behavior. Compare their plotted shapes near the lower and upper observed horsepower values.

6. Smoothing splines and predictive performance

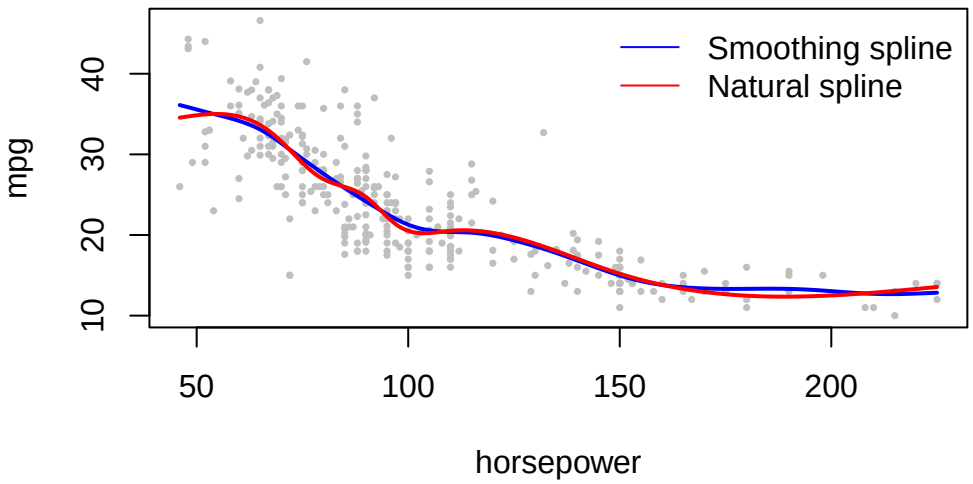
Fit a smoothing spline for `mpg` versus `horsepower`. Select its smoothing parameter using cross-validation or generalized cross-validation (GCV), and report its effective degrees of freedom.

Plot the smoothing spline alongside the selected natural spline from Part 5. Then use the held-out test set to compare the following models, all using `horsepower` alone:

(a) linear regression; (b) CV-selected polynomial; (c) cubic regression spline with three specified interior knots; (d) CV-selected natural spline; (e) smoothing spline.

Report the test MSEs in a single table. Do not use test errors to retune the models.

```
# GCV selects smoothing automatically from training data only.
smooth_fit <- smooth.spline(auto_train$horsepower,auto_train$mpg,cv=FALSE)
smooth_df <- smooth_fit$df
plot(mpg ~ horsepower,auto_train,pch=16,cex=.5,col="gray75")
lines(hp_grid$horsepower,predict(smooth_fit,hp_grid$horsepower)$y,
      col="blue",lwd=2)
lines(hp_grid$horsepower,predict(ns_fit,hp_grid),col="red",lwd=2)
legend("topright",c("Smoothing spline","Natural spline"),
       col=c("blue","red"),lty=1,bty="n")
```




```
auto_results <- data.frame(
  Model=c("Linear","CV polynomial","Cubic regression spline",
          "Natural spline","Smoothing spline"),
  Test_MSE=c(mse(auto_test$mpg,predict(auto_linear,auto_test)),
             mse(auto_test$mpg,predict(auto_poly,auto_test)),
             mse(auto_test$mpg,predict(bs_fit,auto_test)),
             mse(auto_test$mpg,predict(ns_fit,auto_test)),
             mse(auto_test$mpg,predict(smooth_fit,auto_test$horsepower)$y)))
print(auto_results,row.names=FALSE)
```

```
                  Model Test_MSE
                 Linear 24.00307
          CV polynomial 18.17276
 Cubic regression spline 18.40418
         Natural spline 18.93284
       Smoothing spline 19.11010
```

```
cat("Smoothing spline effective df:",round(smooth_df,3),"\n")
```

```
Smoothing spline effective df: 8.957
```

Q14. How does a smoothing spline control flexibility? Compare the fitted curves, effective degrees of freedom where applicable, and predictive errors. Which differences are most noticeable?

> A smoothing spline controls flexibility by penalizing integrated squared curvature, $\lambda \int (f''(t))^2 dt$; GCV selects the smoothing parameter using training data. Its effective degrees of freedom are **8.96**, whereas the natural spline uses **7** basis degrees of freedom (plus an intercept). Both curves capture nonlinear changes in MPG with horsepower, though their local curvature and boundary behavior can differ. The lowest observed test MSE is from **CV polynomial** (18.173). The difference between the smoothing and natural spline test MSEs is **0.177**; all comparisons use the same held-out test set without retuning.

7. Generalized additive models

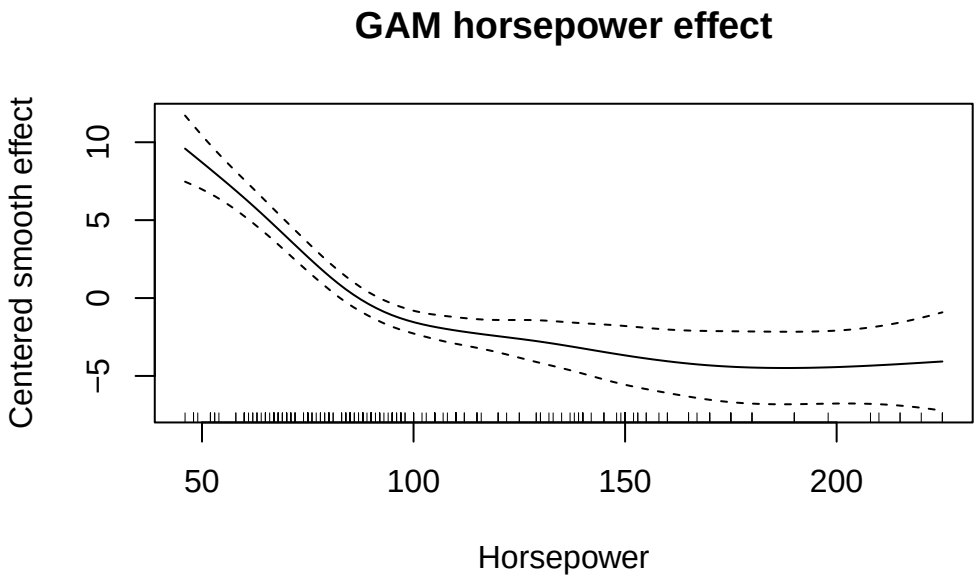
Continue using the `Auto` dataset and the same training/test split. Now use three predictors: `horsepower`, `year`, and `cylinders`.

Fit (a) an ordinary multiple linear regression and (b) a generalized additive model (GAM):

$$E[Y \mid X] = \beta_0 + s(\text{horsepower}) + \beta_1\,\text{year} + \alpha_{\text{cylinders}},$$

where s is a smooth function and `cylinders` is treated as categorical in *both* models. Use a suitable method to select the smoothness of s from the training data. Plot the estimated smooth effect of `horsepower` with approximate uncertainty bands. Compare test MSEs for the two models.

```
auto_train$cylinders <- factor(auto_train$cylinders)
auto_test$cylinders <- factor(auto_test$cylinders,
                              levels=levels(auto_train$cylinders))
mlr_fit <- lm(mpg ~ horsepower + year + cylinders,data=auto_train)
# REML estimates the horsepower smoothness from the training set.
gam_fit <- gam(mpg ~ s(horsepower,bs="cr",k=10) + year + cylinders,
               data=auto_train,method="REML")
plot(gam_fit,select=1,shade=TRUE,shade.col="gray85",
     xlab="Horsepower",ylab="Centered smooth effect",main="GAM horsepower effect")
```




```
gam_results <- data.frame(
  Model=c("Multiple linear regression","GAM"),
  Test_MSE=c(mse(auto_test$mpg,predict(mlr_fit,auto_test)),
             mse(auto_test$mpg,predict(gam_fit,auto_test))))
print(gam_results,row.names=FALSE)
```

```
                     Model  Test_MSE
 Multiple linear regression 11.833627
                        GAM  9.990549
```

```
print(summary(gam_fit)$s.table)
```

```
                   edf   Ref.df        F p-value
s(horsepower) 4.758209 5.805855 24.73092       0
```

Q15. What does the smooth effect of `horsepower` suggest about the relationship with `mpg`, after adjusting for `year` and `cylinders`? Does the GAM improve test MSE compared with the multiple linear model?

> The estimated horsepower effect is nonlinear after adjusting for year and categorical cylinders; interpret its shape from the centered smooth plot, not as an unadjusted relationship. The GAM improves held-out test MSE relative to the multiple linear model.

8. Takeaways

Q16. We have used polynomial degree, ridge and lasso penalties, and spline smoothing parameters to control fitted functions. Briefly explain how these approaches balance model flexibility, estimation variance, and predictive performance.

> Polynomial degree and regression-spline knot counts change the dimension of the fitted function space; ridge/lasso and smoothing splines instead control flexibility through penalties. Greater flexibility can reduce bias but increase estimation variance, so training-only CV or GCV is used to tune complexity for prediction.